ANATOMY OF A BROWSER

Embedded Mobile Lizards

David Llewellyn-Jones
5th November 2024

GECKO

1. Mozilla's rendering engine used in Firefox
2. Released in 2000, Netscape 6
3. Alternative to Blink, WebKit, Netsurf

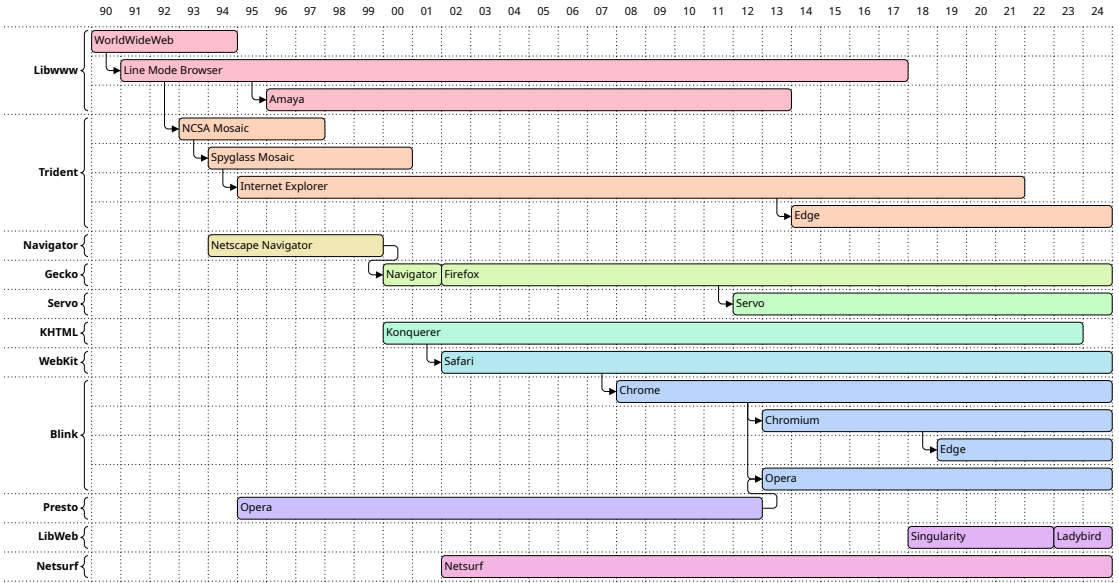
EMBEDDED BROWSERS

> *For many of us, a browser is an application... You click an icon on your graphical operating system, navigate somewhere with a URL bar, search, and so on... In contrast, an* ***embedded browser*** *is contained within another application or is built for a specific purpose and runs in an embedded system, and the application controlling the embedded browser does not provide all the typical features of browsers running in desktops.*
>
> *WPE, October 2024*

EMBEDDED BROWSERS

What does “embedded” mean?

1. Smaller resource footprint devices
2. Embeddable in other programs
3. Minimal user interface
4. All of the above

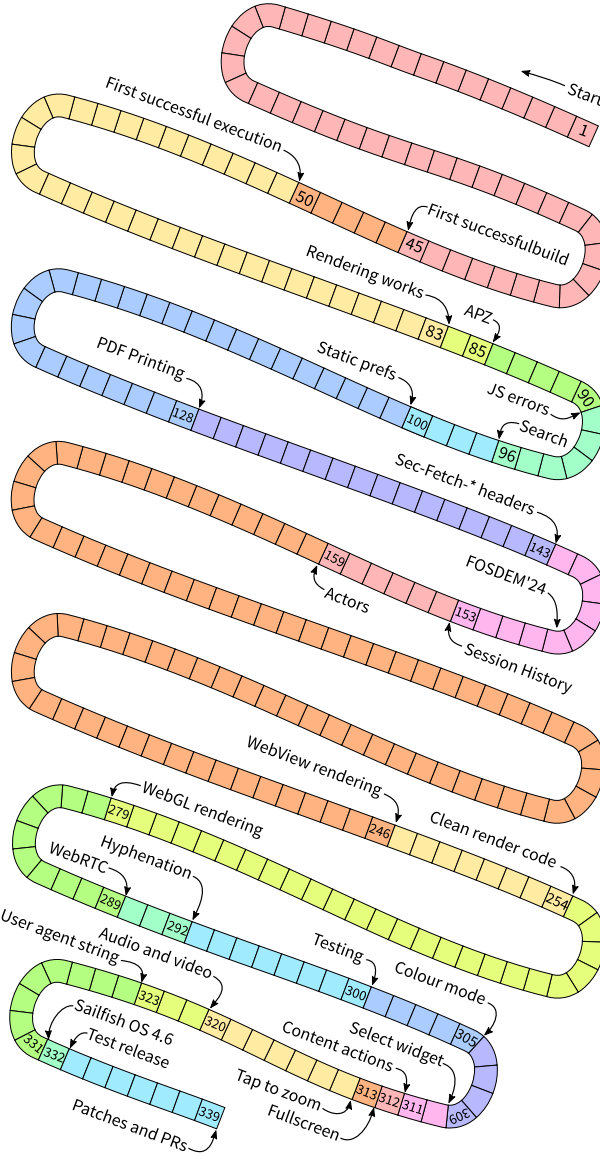

90 91 92 93 94 95 96 97 98 99 00 01 02 03 04 05 06 07 08 09 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
Libwww
WorldWideWeb
Line Mode Browser
Amaya
Trident
NCSA Mosaic
Spyglass Mosaic
Internet Explorer
Edge
Navigator
Netscape Navigator
Gecko
Navigator
Firefox
Servo
Servo
KHTML
Konquerer
WebKit
Safari
Blink
Chrome
Chromium
Edge
Opera
Presto
Opera
LibWeb
Singularity
Ladybird
Netsurf
Netsurf

EMBEDDING GECKO

> *Gecko has limited embedding capability that is not well-documented, not well-maintained and not heavily invested in... We have at various points in history had embedding APIs/capabilities, but we have either dropped them (gtkmozembed) or let them bit-rot (IPCLite).*
>
> *Chris Lord, February 2016*

CONTEXT

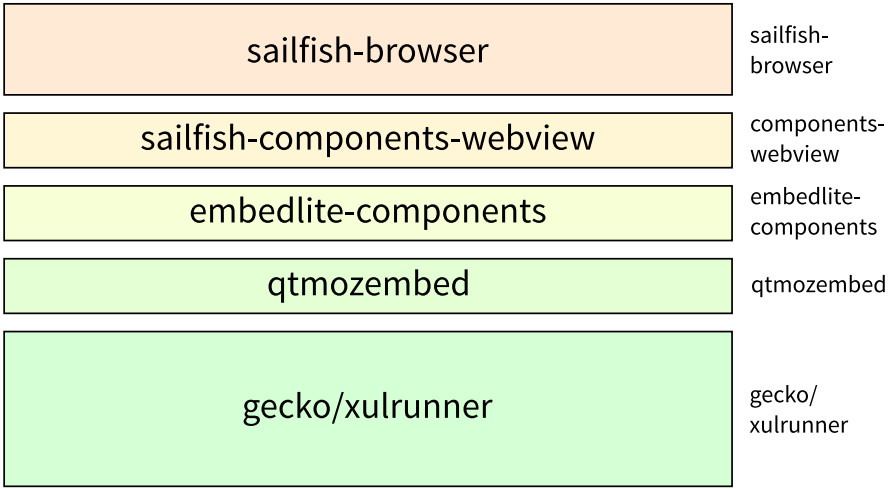
1. Work upgrading the browser on Sailfish OS
2. ESR 78.15.0 to ESR 91.9.1
3. Work conducted over 13 months: August 2023 - September 2024


Start
1
First successful execution
50
First successfulbuild
45
Rendering works
83
APZ
85
PDF Printing
128
Static prefs
100
JS errors
90
Search
96
Sec-Fetch-* headers
143
FOSDEM'24
153
Actors
159
Session History
WebView rendering
246
Clean render code
254
WebGL rendering
279
Hyphenation
292
WebRTC
289
Testing
300
Colour mode
305
Audio and video
320
User agent string
323
Select widget
309
Content actions
311
312
313
Tap to zoom
Fullscreen
Sailfish OS 4.6
332
Test release
333
Patches and PRs
339

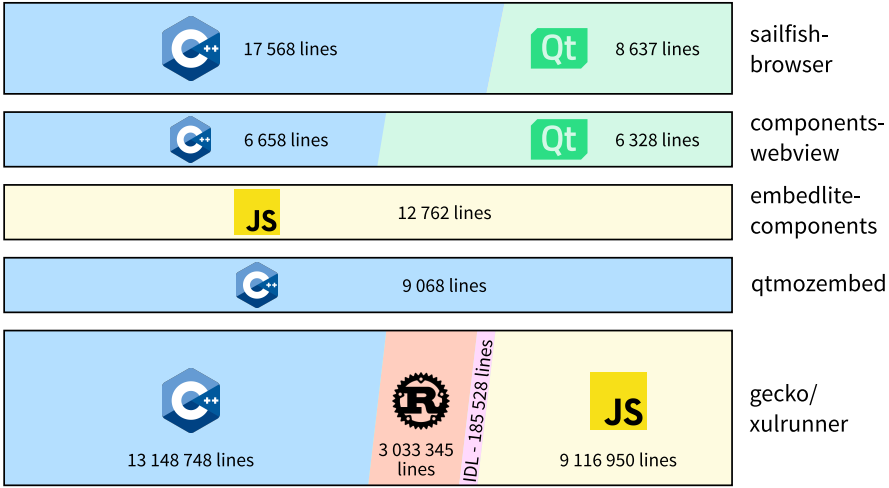
CHALLENGES WORKING WITH THE BROWSER

1. Long build times
2. Complex build process
3. Random upstream changes (e.g. converting quote characters)
4. Removal of the entire EmbedLite rendering pipeline

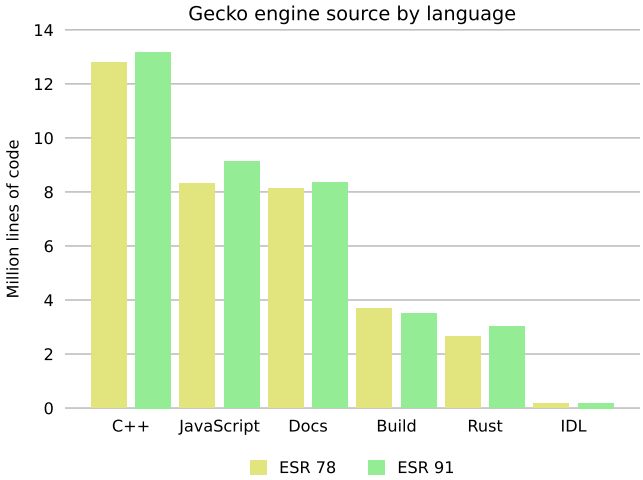
SAILFISH BROWSER



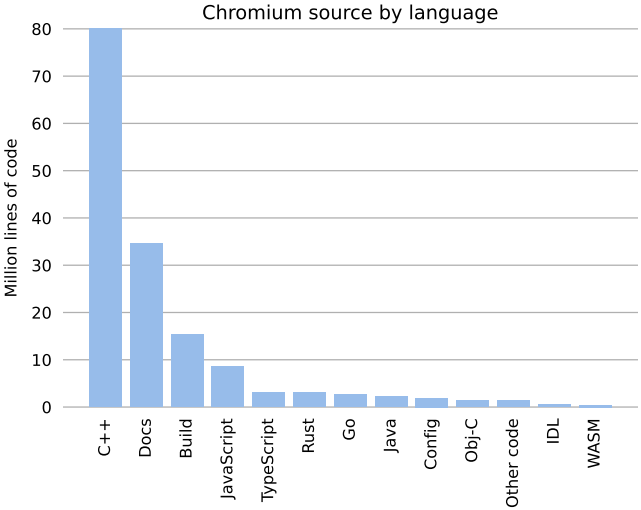
SAILFISH BROWSER



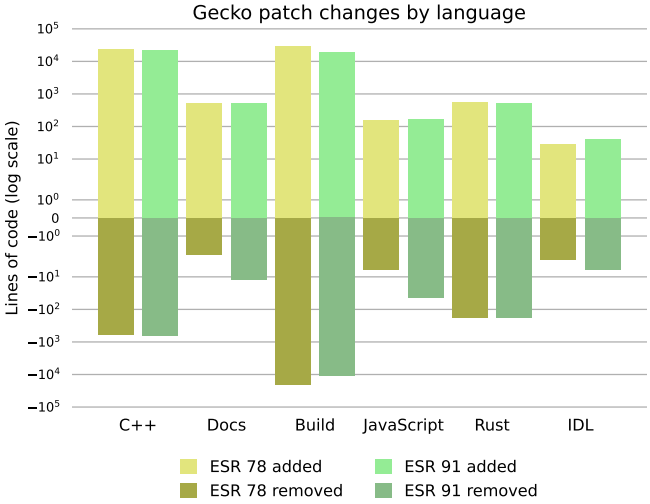
GECKO CODE DISTRIBUTION



CHROMIUM CODE DISTRIBUTION



GECKO PATCHES

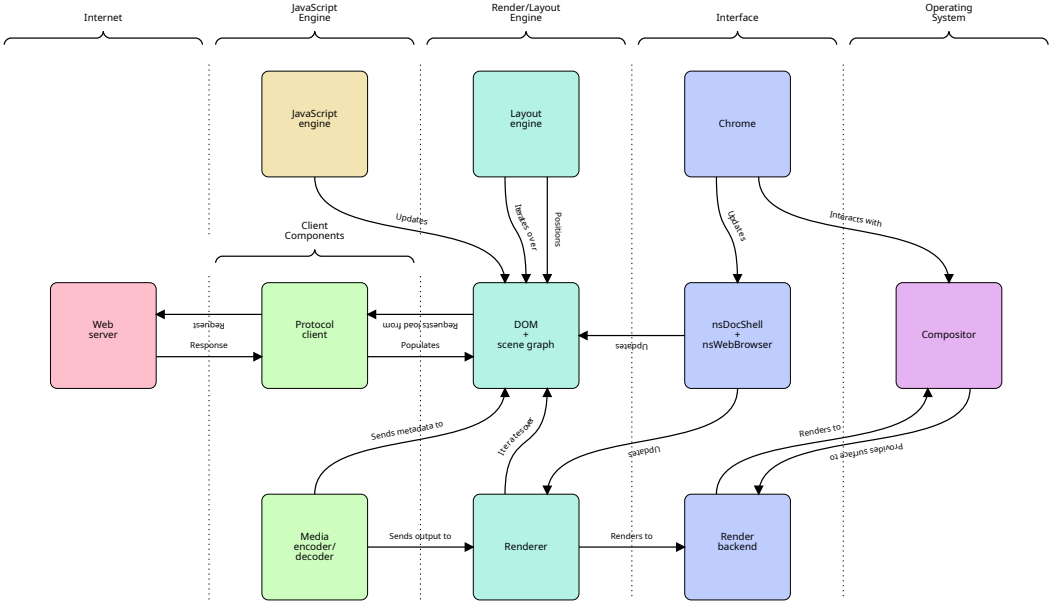


DEV DIARY

https://
www.flypig.co.uk
/gecko

BASIC BROWSER STRUCTURE

1. Protocol client (HTTP/S, file, FTP,...)
2. DOM - Document Object Model
3. JavaScript engine
4. Media decoder (JPG, PNG, audio, video)
5. Layout/rendering engine (HTML, CSS, SVG)
6. User interface


Internet
JavaScript Engine
Render/Layout Engine
Interface
Operating System
JavaScript engine
Layout engine
Chrome
Updates
Iterates over
Positions
Updates
Interacts with
Client Components
Web server
Protocol client
DOM + scene graph
nsDocShell + nsWebBrowser
Compositor
Request
Response
Requests load from
Populates
Updates
Sends metadata to
Iterates over
Updates
Renders to
Provides surface to
Media encoder/ decoder
Sends output to
Renderer
Renders to
Render backend

EMBEDDED BROWSER CONCEPTS

1. Embedding API objects/interfaces
2. Event loop and user input (APZ)
3. Message sending/receiving
4. JavaScript in-DOM execution
5. JavaScript privileged execution
6. IDL - interface definition language

EMEDDING APIS

Most browser embedding frameworks offer a similar API

1. Settings controller
2. Javascript execution
3. Web controls (load, forwards, back)
4. Separate view widgets
5. Message passing interface

Brief look at Chromium Embedded Framework, Sailfish WebView, QtWebEngine

```cpp
class CefBrowserHost : public CefBaseRefCounted {
 public:
  static bool CreateBrowser(const CefWindowInfo& windowInfo,
                            CefRefPtr<CefClient> client,
                            const CefString& url,
                            const CefBrowserSettings&
                                  settings,
                            CefRefPtr<CefDictionaryValue>
                                  extra_info,
                            CefRefPtr<CefRequestContext>
                                  request_context);
  CefRefPtr<CefBrowser> GetBrowser();
  void CloseBrowser(bool force_close);
  ...
  void StartDownload(const CefString& url);
  void PrintToPDF(const CefString& path,
                  const CefPdfPrintSettings& settings,
                  CefRefPtr<CefPdfPrintCallback> callback);
  void Find(const CefString& searchText,
                    bool forward,
                    bool matchCase,
                    bool findNext);
  void StopFinding(bool clearSelection);
  bool IsFullscreen();
  void ExitFullscreen(bool will_cause_resize);
  ...
};
```

```cpp
class CefFrame : public CefBaseRefCounted {
 public:
  CefRefPtr<CefBrowser> GetBrowser();
  void LoadURL(const CefString& url);
  CefString GetURL();
  CefString GetName();
  void Cut();
  void Copy();
  void Paste();
  void ExecuteJavaScript(const CefString& code,
                         const CefString& script_url,
                         int start_line);
  void SendProcessMessage(CefProcessId target_process,
                          CefRefPtr<CefProcessMessage>
                                message);
  ...
};
```

```cpp
class CefBrowser : public CefBaseRefCounted {
 public:
  bool CanGoBack();
  void GoBack();
  bool CanGoForward();
  void GoForward();
  bool IsLoading();
  void Reload();
  void StopLoad();
  bool HasDocument();
  CefRefPtr<CefFrame> GetMainFrame();
};
```

```qt
class QWebEnginePage : public QObject
{
    Q_PROPERTY(QUrl requestedUrl...)
    Q_PROPERTY(qreal zoomFactor...)
    Q_PROPERTY(QString title..)
    Q_PROPERTY(QUrl url READ...)
    Q_PROPERTY(bool loading...)
    ...

public:
    explicit QWebEnginePage(QObject *parent);

    virtual void triggerAction(WebAction action,
            bool checked);

    void findText(const QString &subString,
            FindFlags options, const std::function<void(const
            QWebEngineFindTextResult &)> &resultCallback));
    void load(const QUrl &url);
    void download(const QUrl &url, const QString &filename);
    void runJavaScript(const QString &scriptSource,
            const std::function<void(const QVariant &)> &
            resultCallback);
    void fullScreenRequested(QWebEngineFullScreenRequest
            fullScreenRequest);
    ...
};
```

```qt
class QWebEngineView : public QWidget
{
    Q_PROPERTY(QString title...)
    Q_PROPERTY(QUrl url...)
    Q_PROPERTY(QString selectedText...)
    Q_PROPERTY(bool hasSelection...)
    Q_PROPERTY(qreal zoomFactor...)
    ...

public:
    explicit QWebEngineView(QWidget *parent);
    QWebEnginePage *page() const;
    void setPage(QWebEnginePage *page);

    void load(const QUrl &url);
    void findText(const QString &subString,
            FindFlags options, const std::function<void(const
            QWebEngineFindTextResult &)> &resultCallback);
    QWebEngineSettings *settings() const;
    void printToPdf(const QString &filePath, const
            QPageLayout &layout, const QPageRanges &ranges);
    ...

public slots:
    void stop();
    void back();
    void forward();
    void reload();
    ...
};
```

```cpp
class WebEngine
{
public:
    static WebEngine *instance();
    ...

public slots:
    void addObserver(const QString &aTopic);
    void removeObserver(const QString &aTopic);
    void notifyObservers(const QString &topic, const QString &value);
    void addComponentManifest(const QString &manifestPath);
    void setProfile(const QString &);
    ...

public signals:
    void recvObserve(const QString message, const QVariant data);
    ...
};
```

```cpp
class WebEngineSettings
{
    Q_PROPERTY(bool autoLoadImages...)
    Q_PROPERTY(bool javascriptEnabled...)
    Q_PROPERTY(bool popupEnabled...)
    Q_PROPERTY(CookieBehavior cookieBehavior...)
    ...

public:
    static WebEngineSettings *instance();
    ...
};
```

```cpp
class QuickMozView : public QQuickItem
{
    Q_PROPERTY(QUrl url...)
    Q_PROPERTY(QString title...)
    Q_PROPERTY(bool canGoBack...)
    Q_PROPERTY(bool canGoForward...)
    Q_PROPERTY(int loadProgress...)
    Q_PROPERTY(QString httpUserAgent...)
    ...

public:
    void runJavaScript(const QString &script, const QJSValue &callback, const QJSValue &errorCallback);
    void load(const QString&, const bool& fromExternal);
    void reload();
    void stop();
    void goBack();
    void goForward();
    void loadHtml(const QString &html, const QUrl &baseUrl);
    ...
};
```

INTERFACE DEFINITION LANGUAGE

1. Interaction between native code and JavaScript
2. Requires type conversion and memory management
3. IDL - generate native/JS-compatible interfaces
4. Make native calls from JavaScript
5. Make JavaScript calls from native

```
/**
 * This interface allows creating various prompts that have a specific parent.
 */
[scriptable, uuid(2803541c-c96a-4ff1-bd7c-9cb566d46aeb)]
interface nsIPromptFactory : nsISupports
{
  /**
   * Returns an object implementing the specified interface that creates
   * prompts parented to aParent.
   */
  void getPrompt(in mozIDOMWindowProxy aParent, in nsIIDRef iid,
                 [iid_is(iid),retval] out nsQIResult result);
};
```

```
class NS_NO_VTABLE nsIPromptFactory : public nsISupports {
 public:

  NS_DECLARE_STATIC_IID_ACCESSOR(NS_IPROMPTFACTORY_IID)

  /* Used by ToJSValue to check which scriptable interface is implemented. */
  using ScriptableInterfaceType = nsIPromptFactory;

  /* void getPrompt (in mozIDOMWindowProxy aParent, in nsIIDRef iid, [iid_is (iid), retval] out nsQIResult result); */
  JS_HAZ_CAN_RUN_SCRIPT NS_IMETHOD GetPrompt(mozIDOMWindowProxy *aParent, const nsIID & iid, void * * result) = 0;

};
```

```
function LoginManagerPromptFactory() {
  Services.obs.addObserver(this,
    "quit-application-granted", true);
  ...
}

LoginManagerPromptFactory.prototype = {
  classID : Components.ID("{72de694e-6c88-11e2-a4ee-...}"),
  QueryInterface: ChromeUtils.generateQI([
    Ci.nsIPromptFactory,
    Ci.nsIObserver,
    Ci.nsISupportsWeakReference,
  ]),

  observe : function (subject, topic, data) {
    if (topic == "quit-application-granted") {
      this._cancelPendingPrompts();
      ...
    }
  },

  getPrompt : function (aWindow, aIID) {
    var prompt = new LoginManagerPrompter()
                        .QueryInterface(aIID);
    prompt.init(aWindow, this);
    return prompt;
  },
  ...
}; // end of LoginManagerPromptFactory implementation
```

```
nsresult nsPromptFactory::Init() {
  AddObserver(this, "quit-application-granted", true);
  ...
}

NS_IMETHODIMP
nsPromptFactory::Observe(nsISupports* subject,
                    const char* topic, const char16_t* data) {
  if (!strcmp(topic, "quit-application-granted")) {
    this.CancelPendingPrompts();
    ...
  }

  return NS_OK;
}

NS_IMETHODIMP
nsPromptFactory::GetPrompt(mozIDOMWindowProxy *aParent,
                        const nsIID & iid, void * * result) {
  nsCOMPtr<nsISupports> login = new LoginManagerPrompter();
  nsCOMPtr<nsILoginManagerPrompter> prompt = login
                                .QueryInterface(iid, prompt);
  if (prompt) {
    prompt.Init(aParent, this);
  }

  *aResult = prompt.forget().take();
  return NS_OK;
}
```

```
getPrompt(domWin, iid) {
  if (iid.equals(Ci.nsIAuthPrompt2) || iid.equals(Ci.nsIAuthPrompt)) {
    try {
      let pwmgr = Cc[
        "@mozilla.org/passwordmanager/authpromptfactory;1"
      ].getService(Ci.nsIPromptFactory);
      return pwmgr.getPrompt(domWin, iid);
    } catch (e) {
      Cu.reportError(
        "nsPrompter: Delegation to password manager failed: " + e
      );
    }
  }
},
```

```
NS_IMETHODIMP
nsWindowWatcher::GetNewPrompter(mozIDOMWindowProxy* aParent,
                                nsIPrompt** aResult) {
  // This is for backwards compat only. Callers should just use the prompt
  // service directly.
  nsresult rv;
  nsCOMPtr<nsIPromptFactory> factory =
      do_GetService("@mozilla.org/prompter;1", &rv);
  NS_ENSURE_SUCCESS(rv, rv);
  return factory->GetPrompt(aParent, NS_GET_IID(nsIPrompt),
                            reinterpret_cast<void**>(aResult));
}
```

XULRUNNER

What is xulrunner?

1. XUL: XML User interface Language
2. xulrunner: build process to generate a library
3. Officially deprecated in 2016
4. Outputs libxul.so and related artefacts

EMBEDLITE

What is xulrunner?

1. XULRunner with hooks for Qt event loop support
2. Run as a thread or a process

LIBXUL.SO

1. Provides HTTP client, renderer, JavaScript interpretter
2. Everything except the front-end
3. Methods for sending and receiving IPC calls
4. Exposes the full Gecko interface
5. Including calls to register privileged JavaScript

DOCSHELL

1. -

DOMWINDOW

1. -

FURTHER INFO

Dev Diary	https://www.flypig.co.uk/gecko
Slides source	https://github.com/llewelld/techtalk-gecko-dev
Gecko source	https://github.com/llewelld/gecko-dev
CEF	https://bitbucket.org/chromiumembedded/cef
QtWebEngine	https://doc.qt.io/qt-6/qtwebengine-index.html
WebView	https://sailfishos.org/develop/docs/sailfish-components-webview